

ОМСК 2026

## СОДЕРЖАНИЕ

|                                                         |    |
|---------------------------------------------------------|----|
| Введение . . . . .                                      | 2  |
| Основная часть . . . . .                                | 3  |
| 1 Модели машинного обучения и их сериализация . . . . . | 3  |
| 1.1 Выбор датасета и постановка задачи . . . . .        | 3  |
| 1.2 Состав моделей . . . . .                            | 3  |
| 1.3 Инструменты сериализации . . . . .                  | 4  |
| 2 Разработка веб-дашборда . . . . .                     | 4  |
| 2.1 Структура приложения . . . . .                      | 4  |
| 2.2 Фрагменты исходного кода . . . . .                  | 5  |
| 2.3 Размещение исходников и публикация . . . . .        | 5  |
| 3 Демонстрация результатов прогнозирования . . . . .    | 5  |
| 3.1 Корректные (типичные) данные . . . . .              | 5  |
| 3.2 Данные с выбросами и редкими режимами . . . . .     | 6  |
| Заключение . . . . .                                    | 8  |
| Список использованных источников . . . . .              | 9  |
| Приложение А . . . . .                                  | 10 |
| Приложение Б . . . . .                                  | 12 |

## ВВЕДЕНИЕ

Актуальность темы обусловлена широким внедрением методов машинного обучения (МЛ) в промышленную аналитику и прогностическое обслуживание оборудования. После этапа обучения практический интерес представляет *инференс* - воспроизводимое получение прогнозов на новых данных с понятной интерпретацией для пользователя.

Для реализации работы использован стек технологий на языке Python: библиотека `scikit-learn` (классические и ансамблевые модели, сериализация через `pickle`), `CatBoost` (нативное сохранение модели), `TensorFlow / Keras` (полносвязная нейронная сеть), `pandas` и `NumPy` для табличных данных, `Plotly` для интерактивной визуализации, `Streamlit` - для построения многостраничного веб-интерфейса [1, 2, 3, 4].

**Цель работы:** разработать веб-дашборд для инференса шести моделей регрессии и представления результатов анализа датасета AI4I 2020 Predictive Maintenance.

### **Задачи:**

- 1) подобрать и обучить шесть моделей различных классов (классика, бустинг, `CatBoost`, бэггинг, стэкинг, нейросеть), оценить качество по коэффициенту детерминации  $R^2$  и сериализовать модели;
- 2) реализовать дашборд на `Streamlit` с разделами: сведения о разработке, описание данных, визуализации, инференс;
- 3) подготовить репозиторий с документацией и примерами данных для демонстрации прогнозирования (в том числе для оформления отчёта).

## ОСНОВНАЯ ЧАСТЬ

### 1 Модели машинного обучения и их сериализация

#### 1.1 Выбор датасета и постановка задачи

В качестве набора данных выбран открытый датасет *AI4I 2020 Predictive Maintenance* [5]. Он описывает режимы работы оборудования, износ инструмента и индикаторы отказов. В рамках РГР решается задача **регрессии**: по вектору признаков оценивается нормализованная температура процесса `norm_Processtemperature[K]` (значения в отрезке  $[0, 1]$  относительно исходного диапазона температур в кельвинах). Предобработка и разведочный анализ выполнены в ноутбуке `maintaince_dataset_eda.ipynb` (кодирование типа продукта, MinMax-нормализация, инженерия признаков по критериям отказов).

#### 1.2 Состав моделей

По условию задания сформирован набор из шести моделей, охватывающих требуемые классы алгоритмов. Качество на отложенной тестовой выборке фиксируется в файле `models/metadata.json`.

Таблица 1 – Соответствие обозначений ML1--ML6 реализациям и файлам сериализации

| Ключ | Класс модели                                    | Файл на диске                       |
|------|-------------------------------------------------|-------------------------------------|
| ML1  | Ridge в Pipeline со StandardScaler              | ml1_ridge_pipeline.pkl              |
| ML2  | Градиентный бустинг (GradientBoostingRegressor) | ml2_gradient_boosting.pkl           |
| ML3  | CatBoostRegressor                               | ml3_catboost.cbm                    |
| ML4  | Бэггинг над деревьями решений                   | ml4_bagging.pkl                     |
| ML5  | Стэкинг (базовые модели + Ridge)                | ml5_stacking.pkl                    |
| ML6  | Полносвязная сеть Keras + отдельный scaler      | ml6_fcnn.keras, ml6_fcnn_scaler.pkl |

### 1.3 Инструменты сериализации

Для объектов `sklearn` использован модуль `pickle`. Модель CatBoost сохранена вызовом `save_model` в формат `.cbm` [3]. Нейросеть Keras экспортирована в формат `.keras` [4]. В `metadata.json` дополнительно сохранены список признаков, границы для денормализации целевой температуры и MinMax-границы исходных непрерывных признаков для ввода данных в физических единицах в дашборде.

## 2 Разработка веб-дашборда

### 2.1 Структура приложения

Приложение организовано как многостраничный проект Streamlit: точка входа `app.py` (страница 1 - сведения о разработчике и тема РГР), каталог `pages/` содержит модули `2_dataset.py` (описание данных и фрагменты кода предобработки из EDA-ноутбука), `3_visualizations.py`

(интерактивные графики Plotly), `4_inference.py` (выбор модели, загрузка CSV, ручной ввод, вывод прогноза в нормализованном виде и в кельвинах). Общая логика загрузки моделей и предсказания вынесена в `_common.py`.

## 2.2 Фрагменты исходного кода

Ниже приведён фрагмент функции пакетного предсказания (выбор модели по ключу и выравнивание столбцов по `metadata`):

```
def predict_batch(model_key: str, X: pd.DataFrame, meta: dict) -> np.ndarray:
    Xa = align_features(X, meta["feature_columns"])
    if model_key == "ML1":
        return np.asarray(_load_ml1().predict(Xa), dtype=float)
    if model_key == "ML3":
        return np.asarray(_load_ml3().predict(Xa), dtype=float)
    # ... ML2, ML4, ML5, ML6 ...
    raise ValueError(model_key)
```

Фрагмент сериализации CatBoost после обучения:

```
ml3.save_model(str(MODEL_DIR / "ml3_catboost.cbm"))
```

## 2.3 Размещение исходников и публикация

Исходный код РГР размещён в открытом репозитории GitHub: [https://github.com/Nikitososs/rgr\\_ml\\_4\\_sem](https://github.com/Nikitososs/rgr_ml_4_sem). Веб-сервис дашборда опубликован на Streamlit Community Cloud: <https://nikitososs.streamlit.app>.

## 3 Демонстрация результатов прогнозирования

### 3.1 Корректные (типичные) данные

Файл `rgr_examples_typical.csv` содержит три первые записи исходного датасета без индикации отказа: параметры процесса находятся вблизи центра распределений. Такие примеры иллюстрируют ожидаемое поведение моделей в *штатной* области данных.

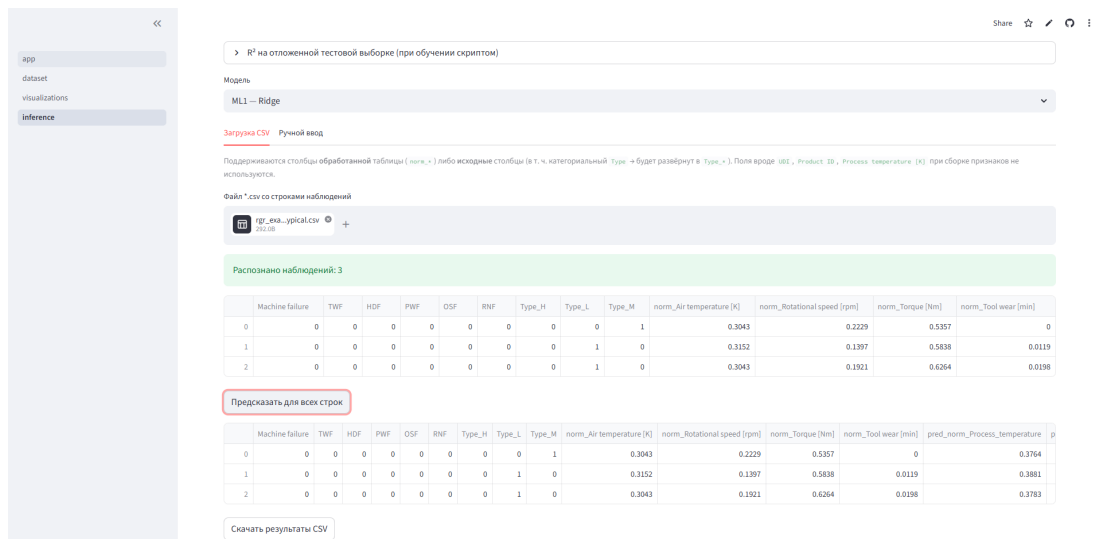


Рисунок 1 – Скриншот страницы инференса для rgr\_examples\_typical.csv. Пример прогнозирования на данных без выбросов

### 3.2 Данные с выбросами и редкими режимами

Файл rgr\_examples\_outliers.csv включает три наблюдения:

- запись с очень высокой скоростью вращения и низким моментом при отказе по мощности (PWF);
- запись с максимальным износом инструмента и комбинацией отказов TWF/OSF;
- запись с моментом на верхней границе диапазона датасета при низкой скорости и отказе PWF.

Share ☆ ↺ ⋮

> R² на отложенной тестовой выборке (при обучении скриптом)

Модель

ML1 — Ridge

Загрузка CSV

Ручной ввод

Поддерживаются столбцы обработанной таблицы (norm\_\*) либо исходные столбцы (в т.ч. категориальный Type → будет развёрнут в Type\_\*). Поля вроде u01, Product ID, Process temperature [K] при сборке признаков не используются.

Файл \*.csv со строками наблюдений

rgr\_examples\_outliers.csv

302.0B

+

Распознано наблюдений: 3

|   | Machine failure | TWF | HDF | PWF | OSF | RNF | Type_H | Type_L | Type_M | norm_Air temperature [K] | norm_Rotational speed [rpm] | norm_Torque [Nm] | norm_Tool wear [min] |        |
|---|-----------------|-----|-----|-----|-----|-----|--------|--------|--------|--------------------------|-----------------------------|------------------|----------------------|--------|
| 0 | 1               | 0   | 0   | 1   | 0   | 0   | 0      | 0      | 1      | 0                        | 0.3913                      | 0.9854           | 0.011                | 0.5652 |
| 1 | 1               | 1   | 0   | 0   | 0   | 1   | 0      | 0      | 0      | 1                        | 0.7935                      | 0.1665           | 0.7005               | 1      |
| 2 | 1               | 0   | 0   | 1   | 0   | 0   | 0      | 0      | 1      | 0                        | 0.5543                      | 0.0186           | 1                    | 0.0119 |

Предсказать для всех строк

|   | Machine failure | TWF | HDF | PWF | OSF | RNF | Type_H | Type_L | Type_M | norm_Air temperature [K] | norm_Rotational speed [rpm] | norm_Torque [Nm] | norm_Tool wear [min] | pred_norm_Process temperature [K] |
|---|-----------------|-----|-----|-----|-----|-----|--------|--------|--------|--------------------------|-----------------------------|------------------|----------------------|-----------------------------------|
| 0 | 1               | 0   | 0   | 1   | 0   | 0   | 0      | 1      | 0      | 0.3913                   | 0.9854                      | 0.011            | 0.5652               | 0.4283                            |
| 1 | 1               | 1   | 0   | 0   | 1   | 0   | 0      | 0      | 1      | 0.7935                   | 0.1665                      | 0.7005           | 1                    | 0.7438                            |
| 2 | 1               | 0   | 0   | 1   | 0   | 0   | 0      | 1      | 0      | 0.5543                   | 0.0186                      | 1                | 0.0119               | 0.5618                            |

Скачать результаты CSV

Рисунок 2 – Скриншот страницы инференса для `rgr_examples_outliers.csv`. Пример прогнозирования на данных с выбросами



## ЗАКЛЮЧЕНИЕ

В ходе выполнения расчётно-графической работы подготовлен конвейер обучения и сериализации шести моделей регрессии для датасета AI4I 2020 Predictive Maintenance, реализован веб-дашборд на библиотеке Streamlit с разделами описания данных, визуализации и инференса, организован открытый репозиторий и подготовлены примеры CSV для демонстрации прогнозов. Полученный инструмент позволяет воспроизводимо сравнивать ответы различных архитектур МЛ на одних и тех же входных данных.

## СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] Pedregosa F. et al. Scikit-learn: Machine Learning in Python // *Journal of Machine Learning Research*. - 2011. - Vol. 12. - P. 2825--2830. - URL: <http://scikit-learn.org/stable/> (дата обращения: 16.05.2026).
- [2] Streamlit documentation. - URL: <https://docs.streamlit.io/> (дата обращения: 16.05.2026).
- [3] Prokhorenkova L. et al. CatBoost: unbiased boosting with categorical features // *Advances in Neural Information Processing Systems*. - 2018. - Vol. 31. - URL: <https://catboost.ai/> (дата обращения: 16.05.2026).
- [4] Abadi M. et al. TensorFlow: Large-Scale Machine Learning on Heterogeneous Distributed Systems. - 2015. - URL: <https://www.tensorflow.org/> (дата обращения: 16.05.2026).
- [5] Mathew L., Twomey K. et al. AI4I 2020 Predictive Maintenance Dataset // UCI Machine Learning Repository. - 2020. - DOI: 10.24432/C5HS5C. - URL: <https://archive.ics.uci.edu/dataset/601/ai4i+2020+predictive+maintenance+dataset> (дата обращения: 16.05.2026).  
URL: <https://pareto.ai/blog/machine-learninginference> (дата обращения: 16.05.2026).

# ПРИЛОЖЕНИЕ А

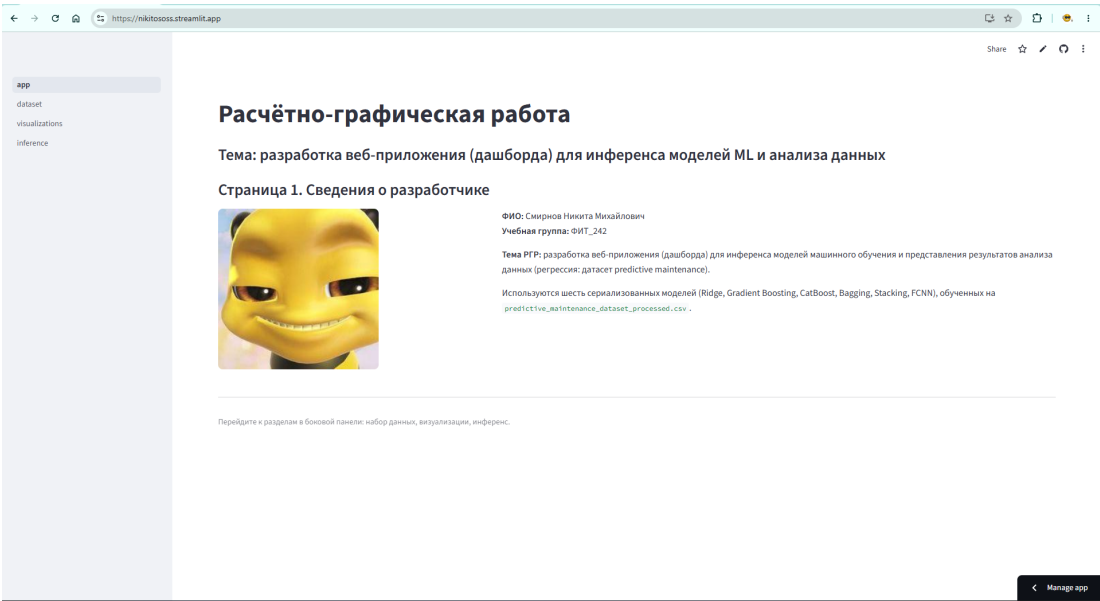


Рисунок 3 – Главная страница дашборда

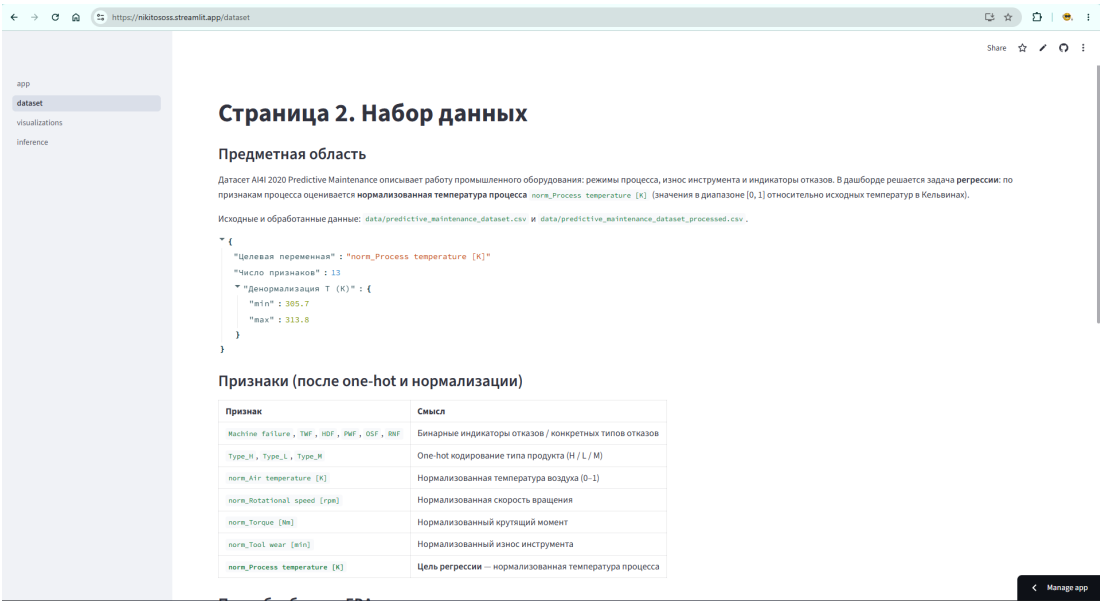


Рисунок 4 – Страница с информацией о датасете

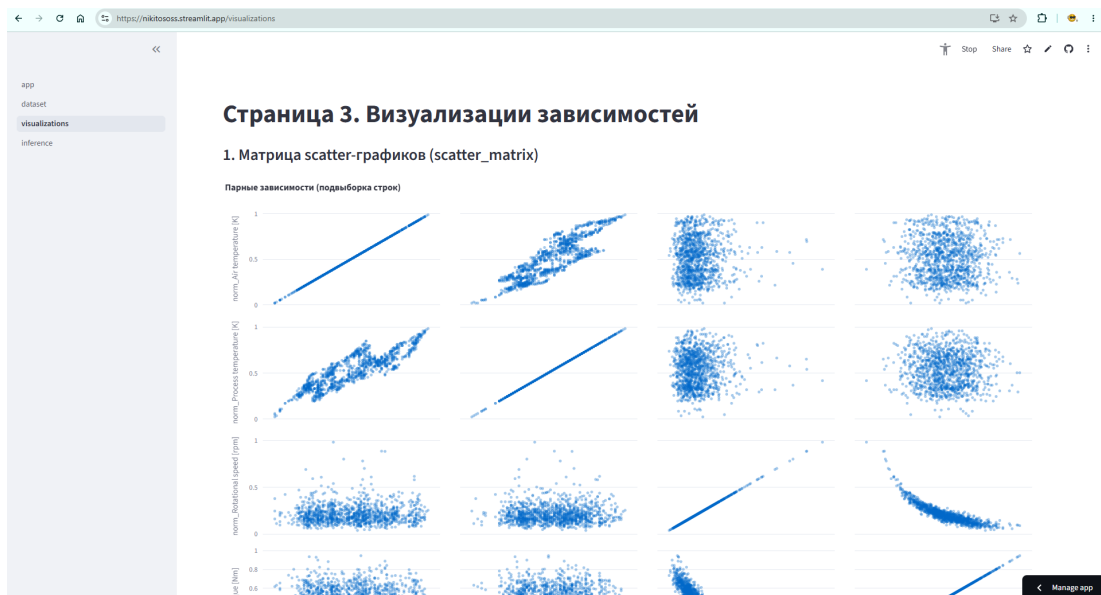


Рисунок 5 – Страница с визуализациями

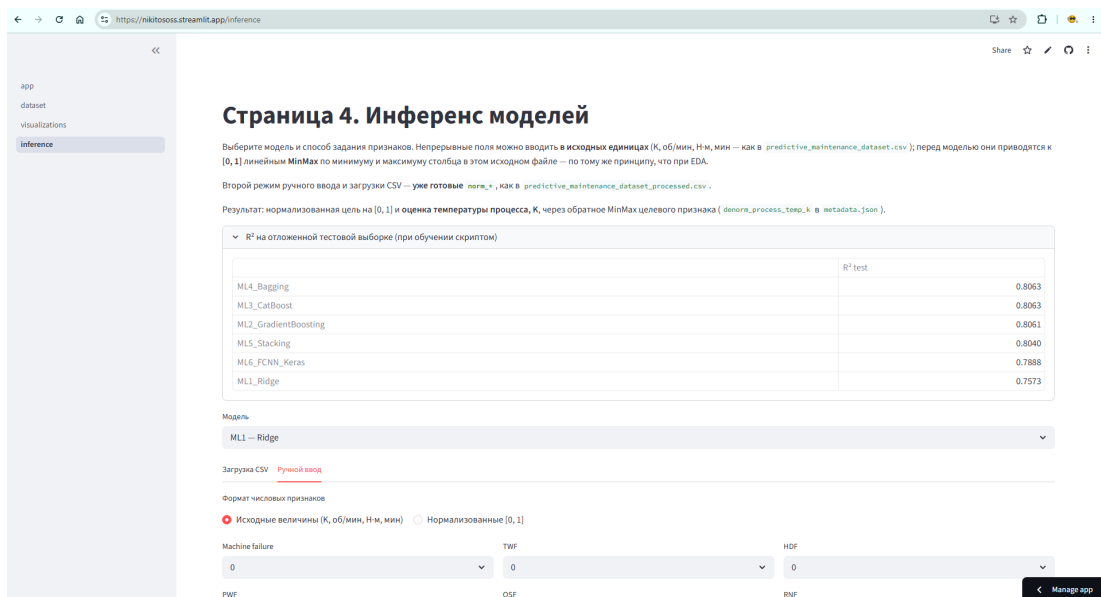


Рисунок 6 – Инференс

## ПРИЛОЖЕНИЕ Б

app.py — точка входа дашборда (стр. 1)

```
import streamlit as st

from _common import PHOTO_PATH

st.set_page_config(
    page_title="РГР – ML Inference",
    page_icon="💎",
    layout="wide",
    initial_sidebar_state="expanded",
)

st.title("Расчётнографическая- работа")
st.subheader(
    "Тема: разработка вебприложения- дашборда() для инференса моделей ML и анализа данных"
)

st.markdown("### Страница 1. Сведения о разработчике")

col1, col2 = st.columns([1, 2])
with col1:
    st.image(str(PHOTO_PATH), width=280)
with col2:
    st.markdown(
        """
        ФИО** Смирнов Никита Михайлович
        Учебная** группа:** ФИТ_242

        Тема** РГР:** разработка вебприложения- дашборда() для инференса
        моделей машинного обучения
        и представления результатов анализа данных регрессия(: датасет
        predictive maintenance).

        Используются шесть сериализованных моделей (Ridge, Gradient Boosting,
        CatBoost, Bagging, Stacking, FCNN),
        обученных на `predictive_maintenance_dataset_processed.csv`.
        """
    )
```

```

    )

st.divider()
st.caption("Перейдите к разделам в боковой панели: набор данных, визуализации,
инференс.")

```

## TrainModels.py — скрипт обучения и сохранения моделей

```

from __future__ import annotations

import json
import pickle
import sys
from pathlib import Path

import numpy as np
import pandas as pd
from catboost import CatBoostRegressor
from sklearn.ensemble import (
    BaggingRegressor,
    GradientBoostingRegressor,
    HistGradientBoostingRegressor,
    RandomForestRegressor,
    StackingRegressor,
)
from sklearn.linear_model import Ridge
from sklearn.metrics import r2_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeRegressor

ROOT = Path(__file__).resolve().parents[1]
DATA_PATH = ROOT / "data" / "predictive_maintenance_dataset_processed.csv"
ORIG_PATH = ROOT / "data" / "predictive_maintenance_dataset.csv"
MODEL_DIR = ROOT / "models"
RANDOM_STATE = 42

TARGET = "norm_Process temperature [K]"

```

```

def feature_columns(df: pd.DataFrame) -> list[str]:
    return [c for c in df.columns if c != TARGET]

def load_xy() -> tuple[pd.DataFrame, pd.Series]:
    df = pd.read_csv(DATA_PATH)
    y = df[TARGET]
    X = df.drop(columns=[TARGET])
    return X, y

def denorm_bounds() -> tuple[float, float]:
    if ORIG_PATH.exists():
        o = pd.read_csv(ORIG_PATH, usecols=["Process temperature [K]"])
        col = o["Process temperature [K]"]
        return float(col.min()), float(col.max())
    return 305.7, 313.8

def physical_feature_bounds(orig: pd.DataFrame) -> dict[str, dict[str, float]]:
    pairs = (
        ("norm_Air temperature [K]", "Air temperature [K]"),
        ("norm_Rotational speed [rpm]", "Rotational speed [rpm]"),
        ("norm_Torque [Nm]", "Torque [Nm]"),
        ("norm_Tool wear [min]", "Tool wear [min]"),
    )
    out: dict[str, dict[str, float]] = {}
    for norm_col, src in pairs:
        s = orig[src].astype(float)
        out[norm_col] = {"min": float(s.min()), "max": float(s.max())}
    return out

def build_keras_model(input_dim: int):
    import tensorflow as tf

    model = tf.keras.Sequential(
        [
            tf.keras.layers.Input(shape=(input_dim,)),
            tf.keras.layers.Dense(128, activation="relu"),

```

```

        tf.keras.layers.Dropout(0.1),
        tf.keras.layers.Dense(64, activation="relu"),
        tf.keras.layers.Dropout(0.1),
        tf.keras.layers.Dense(32, activation="relu"),
        tf.keras.layers.Dropout(0.1),
        tf.keras.layers.Dense(1, activation="linear"),
    ]
)
model.compile(optimizer=tf.keras.optimizers.Adam(1e-3), loss="mse")
return model

def main() -> None:
    MODEL_DIR.mkdir(parents=True, exist_ok=True)

    X, y = load_xy()
    feats = list(X.columns)
    t_min, t_max = denorm_bounds()
    phys_bounds: dict[str, dict[str, float]] = {}
    if ORIG_PATH.exists():
        full_orig = pd.read_csv(ORIG_PATH)
        phys_bounds = physical_feature_bounds(full_orig)

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=RANDOM_STATE
    )

    metrics: dict[str, float] = {}

    ml1 = Pipeline(
        [
            ("scaler", StandardScaler()),
            ("model", Ridge(alpha=1.0, random_state=RANDOM_STATE)),
        ]
    )
    ml1.fit(X_train, y_train)
    metrics["ML1_Ridge"] = float(r2_score(y_test, ml1.predict(X_test)))
    with open(MODEL_DIR / "ml1_ridge_pipeline.pkl", "wb") as f:
        pickle.dump(ml1, f, protocol=pickle.HIGHEST_PROTOCOL)

```



```

ml2 = GradientBoostingRegressor(
    random_state=RANDOM_STATE,
    n_estimators=250,
    max_depth=4,
    learning_rate=0.08,
    subsample=0.9,
)
ml2.fit(X_train, y_train)
metrics["ML2_GradientBoosting"] = float(r2_score(y_test, ml2.predict(X_test)))
with open(MODEL_DIR / "ml2_gradient_boosting.pkl", "wb") as f:
    pickle.dump(ml2, f, protocol=pickle.HIGHEST_PROTOCOL)

ml3 = CatBoostRegressor(
    depth=6,
    iterations=400,
    learning_rate=0.05,
    loss_function="RMSE",
    verbose=False,
    random_seed=RANDOM_STATE,
)
ml3.fit(X_train, y_train, eval_set=(X_test, y_test), verbose=False)
metrics["ML3_CatBoost"] = float(r2_score(y_test, ml3.predict(X_test)))
ml3.save_model(str(MODEL_DIR / "ml3_catboost.cbm"))

ml4 = BaggingRegressor(
    estimator=DecisionTreeRegressor(max_depth=10, random_state=RANDOM_STATE),
    n_estimators=40,
    random_state=RANDOM_STATE,
    n_jobs=-1,
)
ml4.fit(X_train, y_train)
metrics["ML4_Bagging"] = float(r2_score(y_test, ml4.predict(X_test)))
with open(MODEL_DIR / "ml4_bagging.pkl", "wb") as f:
    pickle.dump(ml4, f, protocol=pickle.HIGHEST_PROTOCOL)

ml5 = StackingRegressor(
    estimators=[
        (
            "rf",
            RandomForestRegressor(

```

```

        n_estimators=120, random_state=RANDOM_STATE, n_jobs=-1
    ),
),
(
    "hgb",
    HistGradientBoostingRegressor(
        max_iter=200, random_state=RANDOM_STATE
    ),
),
],
final_estimator=Ridge(alpha=1.0),
n_jobs=-1,
)
ml5.fit(X_train, y_train)
metrics["ML5_Stacking"] = float(r2_score(y_test, ml5.predict(X_test)))
with open(MODEL_DIR / "ml5_stacking.pkl", "wb") as f:
    pickle.dump(ml5, f, protocol=pickle.HIGHEST_PROTOCOL)

scaler = StandardScaler().fit(X_train)
X_tr = scaler.transform(X_train)
X_te = scaler.transform(X_test)

import tensorflow as tf

early = tf.keras.callbacks.EarlyStopping(
    monitor="val_loss", patience=15, restore_best_weights=True
)
ml6 = build_keras_model(X_tr.shape[1])
ml6.fit(
    X_tr,
    y_train.to_numpy(),
    validation_data=(X_te, y_test.to_numpy()),
    epochs=200,
    batch_size=64,
    callbacks=[early],
    verbose=0,
)
y_pred = ml6.predict(X_te, verbose=0).reshape(-1)
metrics["ML6_FCNN_Keras"] = float(r2_score(y_test, y_pred))
ml6.save(MODEL_DIR / "ml6_fcnn.keras")

```

```

with open(MODEL_DIR / "ml6_fcnn_scaler.pkl", "wb") as f:
    pickle.dump(scaler, f, protocol=pickle.HIGHEST_PROTOCOL)

meta = {
    "target": TARGET,
    "feature_columns": feats,
    "denorm_process_temp_k": {"min": t_min, "max": t_max},
    "physical_feature_bounds": phys_bounds,
    "random_state": RANDOM_STATE,
    "test_size": 0.2,
    "metrics_r2_test": metrics,
    "model_files": {
        "ML1": "ml1_ridge_pipeline.pkl",
        "ML2": "ml2_gradient_boosting.pkl",
        "ML3": "ml3_catboost.cbm",
        "ML4": "ml4_bagging.pkl",
        "ML5": "ml5_stacking.pkl",
        "ML6": "ml6_fcnn.keras",
        "ML6_scaler": "ml6_fcnn_scaler.pkl",
    },
}

with open(MODEL_DIR / "metadata.json", "w", encoding="utf-8") as f:
    json.dump(meta, f, ensure_ascii=False, indent=2)

print("Saved models to", MODEL_DIR)
for k, v in sorted(metrics.items(), key=lambda kv: -kv[1]):
    print(f" {k}: R2_test={v:.4f}")

if __name__ == "__main__":
    main()
    sys.exit(0)

```